
Discovery of Sticky and Responsible Markov Options for Frozen-Option Transfer

Yamen Habib

Department of Communication
and Information Technologies
Universitat Pompeu Fabra
Barcelona, Spain
yamen.habib@upf.edu

Dmytro Grytskyy

Department of Communication
and Information Technologies
Universitat Pompeu Fabra
Barcelona, Spain
dmytro.grytskyy@gmail.com

Rubén Moreno-Bote

Serra Hünter Fellow Programme
Department of Communication
and Information Technologies
Universitat Pompeu Fabra
Barcelona, Spain
ruben.moreno@upf.edu

Abstract

A fundamental problem in hierarchical reinforcement learning is finding a small number of reusable and easily composable behaviors that transfer with little or no fine-tuning to solve novel tasks. Option and skill discovery frameworks are two prominent approaches that address this problem. However, option discovery typically focuses on learning specialized behaviors for a training task, leading to a low diversity of behaviors with limited transferability, and unsupervised skill discovery learns diverse behaviors that can be misaligned with downstream tasks. To address these issues, we introduce Task-Agnostic Sticky and Responsible Options (TASRO), an option-learning framework where the high-level controller chooses low-level options using a fully option-switching Markovian structure, options are promoted to be persistent (sticky) and specialized (responsible), and the high-level controller has both task and task-agnostic information, while the low-level options only have task-agnostic information. We evaluate under a strict frozen-option transfer protocol: pretrain options on a goal-reaching training task, freeze them, and relearn only the controller on downstream tasks. Across navigation, locomotion, and manipulation domains, TASRO improves frozen-option transfer over baselines that rely on skill discovery or even when options are exposed with task information.

1 Introduction

Hierarchical reinforcement learning (HRL) seeks a small set of reusable, composable behaviors that can be transferred to new tasks with *little or no fine-tuning* [1–8]. For example, an agent that can crawl, walk, run, and jump in arbitrary directions, for varying durations, and flexibly switch among these behaviors is better equipped to navigate and survive in a complex forest. These behaviors are most useful if they are pretrained and do not need to be relearned or fine-tuned in the forest. However, discovering a small library of behaviors that is genuinely useful for transfer remains difficult: the behaviors should be temporally extended, composable, reusable across related tasks, and diverse enough to support new downstream objectives [1, 9, 10, 7].

Options are a natural starting point for learning reusable behaviors in HRL because they represent temporally extended closed-loop sub-policies selected by a higher-level controller [1]. Subsequent work learned such options end-to-end from task reward, notably the option-critic architecture [11], its continuous-action extension [12], and related hierarchical policy-search approaches [13]. However, when both options and the high-level controller are given the full task information [11, 12], options tend to overfit to the training task, so that high performance in the training task does not imply that the learned options are reusable downstream. In this case, option re-learning and fine-tuning are required [14]. While fine-tuning has its virtues [14], other work uses *frozen-option transfer*, where the option library is kept fixed and only a new controller is trained on the downstream task [10, 15, 9]. This is a valuable empirical test of whether the learned options are genuinely reusable, and a practically relevant one, since deployed systems often cannot afford extensive retraining of low-level behavior. Two major directions for frozen-option transfer are pretraining options via motion imitation or dedicated training tasks and freezing them for transfer [16, 17], or discovering reusable options from multi-task training distributions [10, 15, 9]. Although extremely promising, these approaches require either high quality demonstrations, which may not be available, or learning across many tasks, which can largely impair sample efficiency during learning.

Skill discovery is an alternative to reward-trained options for learning reusable behaviors. This line of work learns skills from intrinsic diversity or coverage objectives [7, 18]. For instance, DIAYN distinguishes skills through visited states [7], while METRA spreads them using metric structure over state space [18]. This can produce broad, task-agnostic behavioral coverage that is useful for downstream transfer, whether via fine-tuning, planning, or frozen reuse. However, because pretraining is unsupervised, the resulting repertoire is optimized for diversity or coverage rather than for task usefulness. For an industrial robot arm, covering the whole reachable state space may not be needed; instead, learning reusable behaviors that solve the task family of interest, such as moving within the workspace, may be enough. Further, learned skills are not trained together, so that the agent has no experience transitioning between them, which explains their low performance in unstable virtual robots such as the Ant [19]. Having this skill-switching experience is critical to flexibly compose skills to solve downstream tasks. Some other work [20] discovers specialized behavioral components by maximizing mixture entropy while minimizing component entropy and supports frozen-component transfer; however, it does not generate temporally extended behaviors, so the learned tokens remain relatively short-lived. See the additional literature review in Appendix A.1.

In this paper we address the problem of frozen-option transfer under a single option-learning training task, and provide the controller with full option-switching awareness to increase downstream flexibility and option composability. We introduce Task-Agnostic Sticky and Responsible Options (TASRO), an HRL framework where (1) the high-level controller chooses low-level options using a fully option-switching Markovian structure (Markov options), (2) options are promoted to be persistent and specialized (sticky and responsible options), and (3) the high-level controller has both task-agnostic and task-dependent information, while the low-level options only have task-agnostic information (information asymmetry). Our main result is to show that TASRO outperforms option-learning baselines (even with options having full task information) and unsupervised skill-discovery baselines across navigation, locomotion, and manipulation domains in frozen-option transfer. We describe the relevance and novelty of features (1-3) in more detail:

(1) *Markov Options*. Both controller and options are aware of what option was used in the previous time step, so that the transition to a new option is state and pre-option dependent (Fig. 1a). In other words, the controller chooses an option based on the option-augmented state (which includes state and option), while the chosen option selects an action based on the same augmented state or a masked version thereof. With Markov options, there is no need to define option initiation and termination sets, a rather sensitive problem in classical option theory that requires regularization in order to avoid option collapse [1]; instead, the controller learns the optimal switching dynamics between options. Moreover, Markovianity provides the agent with full experience on transitions between options, aiding enhanced stability when composing them to solve downstream tasks with unstable agents. Ours is the first fully Markov option framework.

(2) *Sticky and Responsible Options*. To promote option temporal extension, we introduce a sticky prior on the controller, which encourages reselecting the same option across consecutive steps (Fig. 1b). The temporal extension of the option allows the controller to effectively focus planning on a longer time scale, facilitating learning. To promote option diversity and specialization, a responsibility-

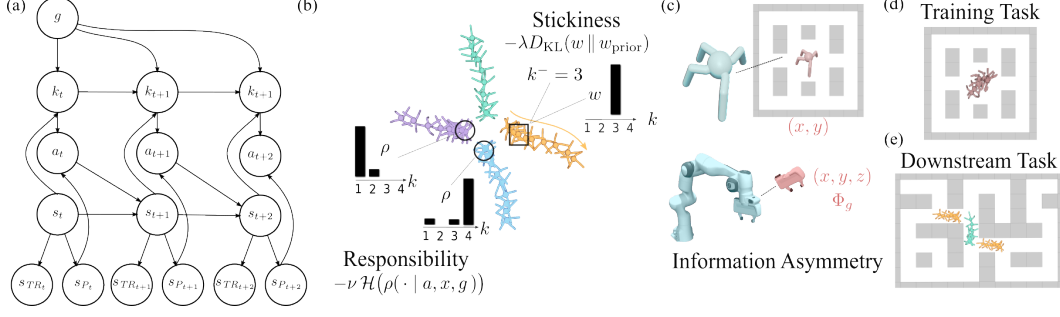


Figure 1: **Overview of TASRO.** (a) **Markovian option process:** The controller selects each option from the goal g , current state s_t , and previous option k_{t-1} , so persistence and switching are Markov decisions on (s_t, k_{t-1}) . The state $s = (s_P, s_{TR})$ is decomposed into task-relevant variables s_{TR} and proprioceptive variables s_P . (b) **Learned option library** ($K = 4$). The sticky-prior KL penalty on the gate distribution $w(k|x, g)$, which selects option k from context x and goal g , encourages reselecting the previous option and thus temporal persistence. The responsibility-entropy penalty term on $\rho(k|x, a, g)$, the posterior probability that option k generated action a , encourages sharper specialization by making each action more confidently attributable to a single option. (c) **Information asymmetry.** Options π_k observe only proprioceptive state s_P and the previously active option k^- . The controller in addition observes the task-relevant state s_{TR} and the goal g . For example, in AntMaze, s_{TR} is the agent’s current xy position and g is the target xy position; in Panda arm manipulation, s_{TR} includes the current gripper xyz position and gripper orientation Φ_g , while g is the desired box pose, defined by its target xyz position and a $6D$ orientation representation. (d) **Training task.** Options are learned on a single goal-conditioned task. (e) **Frozen-option transfer.** The option library is frozen and recomposed by a new controller trained on a downstream task.

weighted regularizer is added to the global objective (Fig. 1b). These and other novel diversity, specialization, and persistence regularizers are described in Eq. 3.

(3) *Information Asymmetry and Frozen-Option Transfer.* To aid frozen-option transferability, during training-task learning we impose an asymmetric information structure across hierarchy levels (Fig. 1c): options receive only task-agnostic state information, while the controller observes task-dependent variables as well [21]. Without this restriction, options can absorb task-specific solutions directly, causing the hierarchy to collapse toward a nearly flat policy whose option identities carry little transferable structure [19]. In our environments, options have only proprioceptive information, such as joint angles, with no information about spatial location in navigation tasks or gripper orientation in manipulation tasks; the controller has full information about all those variables, in addition to goal or reward information. Options are trained jointly with the controller on a single goal-reaching training task (Fig. 1d); on downstream tasks, only the controller is learned, while options are frozen (Fig. 1e).

2 Objective for Sticky and Responsible Markov Options

We aim to learn task-agnostic Markov options to generate diverse, reusable, and easy-to-compose behaviors to solve downstream tasks. We consider a Markov decision process $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$, where $\mathcal{S}$ is a continuous state space, $\mathcal{A}(s)$ is a state-dependent continuous action space, $p(s'|s, a)$ is the transition kernel, $r(s, a)$ is a fixed reward function, and $\gamma \in [0, 1)$ is the discount factor. To introduce information asymmetry, we decompose the state as $s = (s_P, s_{TR})$, where s_P denotes task-agnostic proprioceptive information and s_{TR} denotes task-relevant or reward information.

To make option switching Markovian, we define the option-augmented state $x \equiv (s, k^-) = (s_P, s_{TR}, k^-)$, which corresponds to the state observed by the controller, where $k^- \in \{1, \dots, K\}$ is the option used in the previous time step, and the option state $x_P \equiv (s_P, k^-)$, which corresponds to the state observed by the options. Our framework can also handle the case where options see the full augmented state x , but here we focus on the masked case. We define the mixture policy

$$\pi_m(a|x) = \sum_{k=1}^K w(k|x) \pi_k(a|x_P) \quad (1)$$

as a non-negative linear combination of K option policies $\pi_k(a|x_P)$ with high-level controller $w(k|x) \geq 0$ and $\sum_k w(k|x) = 1$. At each time step, the controller samples an option $k \sim w(\cdot|x)$, the selected option samples an action $a \sim \pi_k(\cdot|x_P)$, the environment transitions according to $p(s'|s, a)$, and the next option-augmented state is $x' = (s', k)$.

Our goal is to optimize both the options $\{\pi_k\}$ and the controller w so as to maximize the value function

$$V(x) = \mathbb{E}_{a_t \sim \pi_m, s_t \sim p} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + R(x_t; \pi, w)) \mid x_0 = x \right]. \quad (2)$$

with the multi-factor regularizer

$$\begin{aligned} R(x; \pi, w) \equiv & \underbrace{\alpha \mathcal{H}(\pi_m(\cdot|x))}_{\text{diversity}} - \underbrace{\beta \mathbb{E}_{k \sim w(\cdot|x)} [\mathcal{H}(\pi_k(\cdot|x_P))]}_{\text{specialization}} - \underbrace{\nu \mathbb{E}_{a \sim \pi_m(\cdot|x)} [\mathcal{H}(\rho(\cdot|x, a))]}_{\text{specialization}} \\ & - \underbrace{\eta \mathcal{H}(w(\cdot|x))}_{\text{persistence}} - \underbrace{\lambda D_{\text{KL}}(w(\cdot|x) \| w_{\text{prior}}(\cdot|x))}_{\text{persistence (stickiness)}}. \end{aligned} \quad (3)$$

Here $\mathcal{H}(\pi(\cdot|y)) = -\sum_a \pi(a|y) \log \pi(a|y)$ is the action entropy of a policy π , $\mathcal{H}(w(\cdot|x)) = -\sum_k w(k|x) \log w(k|x)$ is the entropy of the controller, and $\mathcal{H}(\rho(\cdot|x, a)) = -\sum_k \rho(k|x, a) \log \rho(k|x, a)$ is the entropy of the responsibility function, defined as

$$\rho(k|x, a) = w(k|x) \pi_k(a|x_P) / \pi_m(a|x), \quad (4)$$

which is the probability that a performed action a at option-augmented state x has been generated from option k . We define a sticky prior over options

$$w_{\text{prior}}(k|x = (s, k^-)) = \exp(\Omega_{k^-}^{\text{stick}} \mathbf{1}\{k = k^-\}) / \sum_j \exp(\Omega_{j^-}^{\text{stick}} \mathbf{1}\{j = k^-\}). \quad (5)$$

The goal defined by Eqs. 2,3 thus consists in maximizing the sum of future rewards, while shaping the hierarchy through several regularization terms—the weighting coefficients in Eq. 3 are non-negative. Each of the regularizers plays a role in generating option diversity, specialization, or temporal persistence (stickiness). In order of appearance in Eq. 3: (a) the mixture entropy promotes option diversity by promoting diverse action paths; (b) the negative option entropy promotes intra-option specialization, as the entropy of each policy should be small and thus focused on a small range of actions; (c) the negative option responsibility entropy also favors option specialization, as this entropy is small if just a small range of actions is consistent with choosing a specific option; (d) the negative controller entropy induces temporal persistence of the previously active option; and (e) the negative KL divergence promotes reusing the previously active option through the sticky prior in Eq. 5, and therefore it also produces temporal persistence.

Using mixture entropy to promote diversity has been used before [20], but not in the option framework. Negative option responsibility entropy has also been used before [13] in the option framework, but this term alone without the mixture entropy term induces option collapse to a single flat policy. The other terms are, to our knowledge, novel in the context of option discovery. We also note that the degree of option diversity is shaped by the information asymmetry between the controller and the options: because options have only partial information, they cannot specialize to the training task.

Eq. 2 can be recursively expressed in the form of the Bellman equation

$$V(x) = R(x; \pi, w) + \sum_k w(k|x) \int da \pi_k(a|x_P) Q(x, k, a) \quad (6)$$

$$Q(x, k, a) \equiv r(s, a) + \gamma \int ds' p(s'|s, a) V(x'), \quad x' = (s', k). \quad (7)$$

where $V(x')$ denotes the value at the next option-augmented state $x' = (s', k)$. Here, $Q(x, k, a)$ is the option-action value of choosing option k and action a at option-augmented state x . It includes the immediate reward and then evaluates the continuation at the next option-augmented state $x' = (s', k)$,

so future keep-or-switch decisions remain part of the value function. Therefore, the objective can be expressed as maximizing the value function over option policies $\pi = \{\pi_1, \dots, \pi_K\}$ and the controller w .

We note that although the multi-factor regularization function R defined in Eq. 3 has five parameters, only four parameters are actually independent, as the function can be written as

$$R(x; \pi, w) = \phi_1 \mathbb{E}_{k \sim w(\cdot|x)} [\mathcal{H}(\pi_k(\cdot|x_P))] - \phi_2 \mathbb{E}_{a \sim \pi_m(\cdot|x)} [\mathcal{H}(\rho(\cdot|x, a))] \\ + \phi_3 \mathcal{H}(w(\cdot|x)) + \lambda \sum_k w(k|x) \log w_{\text{prior}}(k|x). \quad (8)$$

with $\phi_1 := \alpha - \beta$, $\phi_2 := \alpha + \nu$, and $\phi_3 := \alpha - \eta + \lambda$; the coefficient λ is the KL weight introduced in Eq. 3.

2.1 Relation between Markov and classical options

Classical options are defined by an initiation set $I_k \subseteq \mathcal{S}$, an option policy π_k^{classic} , and a termination function $\beta_k : \mathcal{S} \rightarrow [0, 1]$ [1]. In the classical framework, the high-level controller is consulted only when the current option terminates. However, this controller has an equivalent per-step formulation: denoting the probability of initiating an option given the state by $\mu(k|s)$, the per-step controller can be written as

$$w_{\text{classic}}(k|s, k^-) = (1 - \beta_{k^-}(s)) \mathbf{1}\{k = k^-\} + \beta_{k^-}(s) \mu(k|s). \quad (9)$$

This induced controller shows how initiation and termination appear in per-step form. Thus, classical call-and-return options factor switching into termination through β_{k^-} and reselection through μ . The reselection policy $\mu(k|s)$ does not condition on the previous option k^- . In contrast, our formulation removes the restricted factorization on Eq. 9 and learns $w(k|s, k^-)$ directly. Thus, in our Markovian framework, $w(k^-|s, k^-)$ is the probability of persistence, and $w(k|s, k^-)$ for $k \neq k^-$ is the probability of switching from k^- to k ; these probabilities do not need to be expressible as Eq. 9. Therefore, the probability of entering option k may depend on the option from which the controller is switching, which is not captured by a single state-only initiation set I_k together with a reselection policy $\mu(k|s)$. Our temporal persistence terms in Eq. 3 favor that the controller has mostly transitions within options and thus high-level control remains sparse, but their transitions are much more flexible than the constrained version in Eq. 9.

The same distinction as the one described before applies to the option policies. Classical option policies have the form $\pi_k^{\text{classic}}(a|s)$, whereas our options may use the option-augmented input, $\pi_k(a|s, k^-)$. Thus both option selection and option execution can condition on the previously active option. The option-critic line of work [11, 12], and much subsequent end-to-end option learning, keep the constrained factorization of Eq. 9. This approximation preserves the classical call-and-return structure, but limits the flexibility of option switching because the reselection policy μ does not condition on k^- . Our controller instead treats keep-or-switch decisions as ordinary Markov decisions on the option-augmented state. This information is important for stable transitions between options, especially in unstable virtual robots.

2.2 Practical training algorithm

This section turns the objective in Eq. 2 into a practical training algorithm. Our algorithm does not optimize Q , π , and w all at once. We instead use a policy-iteration style alternation. At each policy-improvement step, we hold the critic Q and responsibilities ρ fixed. The responsibility ρ is not a separate learned model; it is recomputed from the current controller and option policies using Eq. 4. With these quantities fixed, Proposition 1 in Appendix A.2 lets us update each option independently and then update the controller. Each subproblem is entropy-regularized and concave under $\phi_1 > 0$ for the option updates and $\phi_3 > 0$ for the controller update, so its maximizer is a softmax over actions or options [22]; Appendix A.2 derives the exact forms. In continuous action spaces, their normalizers are action integrals with no closed form, so we approximate them with sampled candidate actions.

Our algorithm is an off-policy actor-critic [23] trained from a replay buffer $\mathcal{D}$ of past transitions [24]. The actors are the option policies $\pi_{\theta,k}$ and the controller w_ψ ; the critics are two independently parameterized Q -networks, Q_{ξ_1} and Q_{ξ_2} . We use the minimum of the two critic values in bootstrap targets to reduce overestimation bias [25]. Slow-moving target copies $\bar{\pi}_k$, $\bar{w}$, and $\bar{Q}_i$, updated by Polyak averaging, stabilize bootstrapping and provide the actions used to estimate the option and

controller targets [26]. Each training iteration updates the critics first, then the option policies, and finally the controller. The objective in Eq. 2 is written for a general reward $r(s, a)$. In our experiments, $g \in \mathcal{G}$ denotes the current goal (e.g., a navigation target or manipulation target), which we make explicit by using the notation $r(s, a, g)$. The controller and critics are goal-conditioned, $w_\psi(k|x, g)$ and $Q_{\xi_i}(x, k, a, g)$, while the option policies remain goal-agnostic, $\pi_{\theta, k}(a|x_P)$. As defined in Section 2, $x = (s, k^-)$ is the option-augmented state and x_P is the masked option input. Thus options learn reusable low-level behaviors from x_P , while the controller and critics use (x, g) to compose options for the current task.

Critic update. The critics provide the value Q used by the option and controller updates. We train the two critics by Bellman regression on replay transitions (x, k, a, r, x', c, g) , where $c \in \{0, 1\}$ is the nonterminal mask. The target networks define the one-step Bellman target

$$\bar{V}(x', g) = \mathbb{E}_{k' \sim \bar{w}, a' \sim \bar{\pi}_{k'}} \left[\min_i \bar{Q}_i(x', k', a', g) \right] + \bar{R}(x', g; \bar{\pi}, \bar{w}), \quad y = r + \gamma c \bar{V}(x', g). \quad (10)$$

Here $\bar{R}(x', g; \bar{\pi}, \bar{w})$ is the regularizer in Eq. 3, estimated under the target networks at the next option-augmented state and current goal. The two critics have separate parameters and independent initializations; using their minimum in $\bar{V}$ reduces overestimation bias, following [25]. Both critics are fit to the same Bellman target by minimizing $\mathcal{L}_Q = \mathbb{E}_{\mathcal{D}} \sum_{i=1}^2 (Q_{\xi_i}(x, k, a, g) - y)^2$.

Option update. For fixed Q , controller w , and responsibilities ρ , each option has its own policy-improvement problem. For each option, this gives a target that puts probability on actions that are high-value under Q and already attributed to option k by ρ . The exact target, denoted by a star, is

$$\pi_k^*(a|x, g) \propto \rho(k|x, a, g)^{\phi_2/\phi_1} \exp(Q(x, k, a, g)/\phi_1). \quad (11)$$

Here $\rho(k|x, a, g)$ is the responsibility from Eq. 4, the posterior probability that option k generated action a . From Eq. 8, ϕ_1 controls how concentrated the option is on high- Q actions, while ϕ_2/ϕ_1 controls how strongly the option specializes on actions for which it is already responsible. Appendix A.2 derives this closed-form target. The target $\pi_k^*(\cdot|x, g)$ is a full-context density, but the parametric option $\pi_{\theta, k}(\cdot|x_P)$ only observes x_P . Fitting one masked-input network to these targets across replay states enforces the information asymmetry during training. We approximate the density in Eq. 11 by N candidate actions sampled from $\bar{\pi}_k(\cdot|x_P)$, assign each candidate an importance weight comparing its target probability to its proposal probability under $\bar{\pi}_k$, and fit $\pi_{\theta, k}$ by weighted likelihood. We sample from $\bar{\pi}_k$ because the option network is iteratively improved toward π_k^* , so its samples concentrate in high-probability regions of the exact target (see Appendix A.2).

Controller update. After option updates, the controller learns to compose them at (x, g) . For fixed Q and analytical option targets, the decomposition in Appendix A.2 assigns each option a score

$$M_k(x, g) := \mathbb{E}_{a \sim \pi_k^*(\cdot|x, g)} [Q(x, k, a, g) + \phi_2 \log \rho(k|x, a, g)] + \phi_1 \mathcal{H}(\pi_k^*(\cdot|x, g)). \quad (12)$$

This score is the option-local objective value at the analytical target, so it summarizes option k 's usefulness at (x, g) . The optimal controller is a softmax over these scores. With $\lambda > 0$, the sticky prior favors reselecting the previous option k^- . The exact controller target is

$$w^*(k|x, g) \propto \exp(M_k(x, g)/\phi_3) w_{\text{prior}}(k|x)^{\lambda/\phi_3}. \quad (13)$$

From Eq. 8, ϕ_3 is the controller softness coefficient: larger values spread probability more broadly across options, while smaller values make the controller more committed to high-score options. The coefficient λ controls the strength of this sticky-prior effect. Appendix A.2 derives this closed-form target. Computing $M_k(x, g)$ exactly is intractable because it requires integrating over all continuous actions for each option. Appendix A.2 shows that this option score can be written as $M_k(x, g) = \phi_1 \log F_k(x, g)$, where $F_k(x, g)$ is the normalizer of the exact option target in Eq. 11. We estimate $F_k(x, g)$ with importance samples from the target option policy $\bar{\pi}_k(\cdot|x_P)$, giving a sampled option score $\widehat{M}_k(x, g)$. We then replace $M_k(x, g)$ by $\widehat{M}_k(x, g)$ in Eq. 13 and fit w_ψ to the resulting sampled controller target by KL minimization.

3 Experiments

Our experiments test whether the learned options can be reused in downstream tasks. We adopt the same frozen-option protocol in every domain: after training-task learning, the option library

is fixed and only a new controller and critic are trained for downstream transfer. The downstream learner selects options proportionally to $\exp(Q(s, g, k^-, k)/\tau)$, with τ tuned automatically [27]. During transfer, the controller holds each frozen option for H primitive steps, preserving temporal abstraction: $H = 10$ for the long-horizon navigation and Barkour tasks, and $H = 5$ for the shorter Panda task. Full environment and training details are deferred to Appendices A.3 and A.4. We choose four domain families because each tests a different kind of reuse (Fig. 6). AntMaze is the easy navigation case: $K = 4$ direction-like options should be enough for a new controller to reach goals in any new environment. HumanoidMaze keeps the same navigation structure but makes the body harder to balance, so reuse requires stable movement as well as route choice. Barkour asks whether a compact $K = 3$ locomotion library trained on a simplified route, based on the Google Barkour v0 model [28] and Barkour benchmark [29], covers more variable downstream waypoints. Panda uses the Franka Emika Panda Description from MuJoCo Menagerie [28] and tests whether options can transfer between standard but different manipulation tasks: $K = 16$ options are trained on pick-and-orient and then recomposed to solve an unseen table-pushing task.

3.1 Baselines and evaluation

We choose baselines to answer five questions about frozen-option transfer. *(1) Does standard option learning transfer well when options see the full task information?* Option-Critic (OC) [11] and PPOC [12] test this case. Both learn options from the training task while giving the options the full state and goal. *(2) Is masking task information from the options enough by itself?* PPOC* tests this case. It uses the same PPO-style option-learning baseline as PPOC, but its options observe only proprioceptive state. This tests whether masking alone is enough, without TASRO’s previous-option conditioning, persistence bias, or responsibility-based specialization. *(3) Does an existing method designed for multitask frozen-option transfer work in our single task training setting?* MSOL [10] tests this case. The original method learns options from multiple tasks and was evaluated with discrete actions. Here we use the same single task training protocol as the other methods, adapt MSOL to continuous control, and freeze its prior goal-agnostic option library for transfer. *(4) Can unsupervised skill discovery produce reusable frozen behaviors?* DIAYN [7] and METRA [18] test this case. *(5) How does transfer compare to learning from scratch in the downstream task?* Flat PPO tests this case. It trains a flat policy directly on the downstream task, without using frozen options. This shows whether frozen options help early in training compared with learning the downstream task from scratch. Appendix A.4 gives the full configurations, adaptations, and training budgets.

3.2 Transfer results

We evaluate frozen-option transfer in the four domains (Fig. 2). HumanoidMaze is the clearest test of option reusability due to the agent’s unstable body dynamics. Here, TASRO largely outperforms all tested methods, including a flat policy, reaching $26.7 \pm 7.9\%$ in the open field (Fig. 2a) and $9.3 \pm 1.4\%$ in the hardest layout (Fig. 2b). This strong performance comes from the diversity and specialization of the trained options (Fig. 3a): when we force a learned option to stay continuously active (denoted forced-option), the obtained trajectories from TASRO are very stable, diverse and specialized, while the other baselines produce less diversity and specialization.

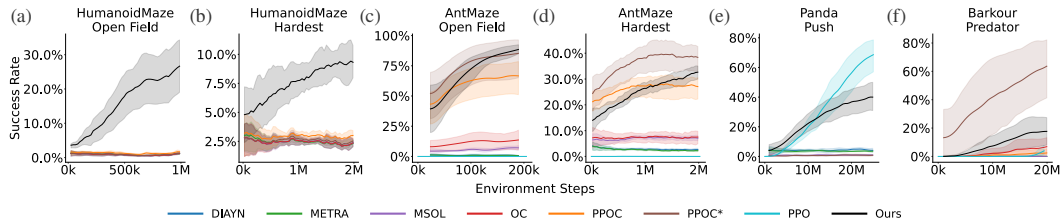


Figure 2: Frozen-option transfer results across four domains. (a,b) HumanoidMaze open field and hardest. (c,d) AntMaze open field and hardest. (e) Panda push. (f) Barkour predator. Mean over 5 agents; 50 randomized episodes per evaluation. Shading denotes 95% bootstrap CIs.

In AntMaze, the quadruped’s greater stability makes navigation more tractable overall. TASRO and PPOC* are now the two strongest frozen-option methods under the main downstream evaluation:

TASRO reaches $88.5 \pm 3.7\%$ in the open field and $32.7 \pm 2.5\%$ in the hardest layout, while PPOC* reaches $85.3 \pm 15.6\%$ and $38.4 \pm 4.7\%$, respectively (Fig. 2c,d). However, forced-option trajectories show that TASRO learns more directionally distinct behaviors (Fig. 3b). A controlled diagnostic with horizon $H = 10$ and episodes capped at 500 rather than the original 1000 steps supports this interpretation: TASRO retains $86.0 \pm 5.5\%$, while PPOC* drops to $55.9 \pm 13.8\%$, consistent with PPOC*'s options relying more on longer interaction horizons than on precise directional specialization. The task-exposed PPOC variant also performs well here, reflecting AntMaze's relative ease compared to the other benchmark domains.

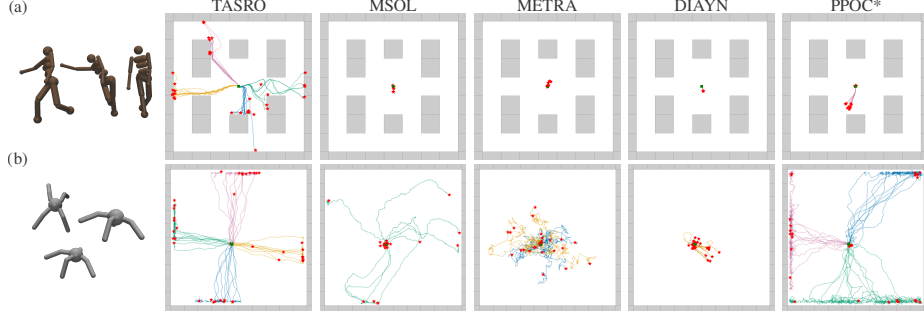


Figure 3: Forced-option trajectories in maze domains. Each option is held active for an entire episode. (a) HumanoidMaze training layout. (b) AntMaze open-field transfer; additional AntMaze seeds are shown in Fig. 7. Red points indicate episode termination (timeout or loss of balance).

Panda arm manipulation tests a different form of reuse: transferring options between two different arm-manipulation objectives. The training task is pick-and-orient and the downstream task is table pushing, but the options see the same proprioceptive view in both tasks. Under frozen-option transfer, TASRO is the only frozen library that makes consistent progress, reaching about $40.0 \pm 9.6\%$ success (Fig. 2e). Flat PPO is the only baseline that does well on this task, reaching $68.6 \pm 10.8\%$ when trained end-to-end, but it is not a frozen-library method. OC/PPOC are omitted here because pick-to-push changes the goal distribution seen by their goal-conditioned options.

Barkour is the main exception to this pattern. With only $K = 3$ options trained on a fixed route, the library appears too compact for the varied headings in the predator downstream task: TASRO reaches $17.7 \pm 9.6\%$ success, while PPOC* reaches $63.6 \pm 19.7\%$ (Fig. 2f). This gap suggests that the training route may not cover the downstream behavioral range, although we cannot rule out other explanations such as $K = 3$ being too small.

Taken together, TASRO is the only frozen-library method that avoids a degenerate option library across all four domain families. PPOC* is the closest competitor, but its options do not support useful transfer in Panda arm manipulation or HumanoidMaze. The forced-option plots give qualitative support for this mechanism. In AntMaze, TASRO produces four distinct directional behaviors in the representative seed, while PPOC* shows partial specialization and the skill-discovery methods are less organized (Fig. 3b). In HumanoidMaze, where unstable balance makes specialization harder, TASRO again shows the most distinct behaviors (Fig. 3a).

3.3 Ablations

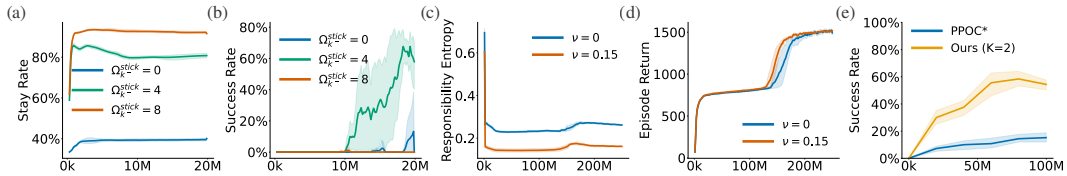


Figure 4: Training-task design checks. (a,b) Barkour stay rate, $\Pr(k = k^-)$, and success across sticky-prior strengths $\Omega_{k^-}^{\text{stick}}$. (c,d) Panda responsibility entropy and return for $\nu = 0$ versus $\nu = 0.15$. (e) AntMaze success with $K = 2$.

Our ablations are designed to check whether TASRO’s design choices produce the intended option structure during training-task learning. Thus, here we do not show downstream transfer results. We use ablations to check four mechanisms: controlling temporal persistence, sharpening option responsibility, hiding task-relevant state from the options, and conditioning options on k^- .

First, the sticky prior should control how often the controller keeps the same option. In Barkour, increasing $\Omega_{k^-}^{\text{stick}}$ increases the stay rate: 40.1% for $\Omega_{k^-}^{\text{stick}} = 0$, close to the 1/3 random baseline for $K = 3$; 80.8% for $\Omega_{k^-}^{\text{stick}} = 4$; and 91.6% for $\Omega_{k^-}^{\text{stick}} = 8$ (Fig. 4a). However, more stickiness is not always better. Barkour reaches 57.9% success at $\Omega_{k^-}^{\text{stick}} = 4$, while both 0 and 8 are at 0% (Fig. 4b).

Second, the responsibility term should sharpen option assignment. In the Panda training task, using $\nu = 0.15$ lowers responsibility entropy from 0.26 to 0.16 (Fig. 4c). Both settings eventually learn the training pick task, but the responsibility-weighted run learns faster and reaches a slightly higher final return (Fig. 4d).

Third, masking s_{TR} should prevent options from encoding the training maze directly. With default option input, AntMaze options observe s_P and k^- , but not xy position. As expected, the forced options look position-blind: when an option is forced into a wall, it keeps pushing rather than navigating the maze (Fig. 5a). When s_{TR} is given to options, they avoid walls directly (Fig. 5b). With $K = 6$, extra options specialize without collapsing (Fig. 5c).

Fourth, while PPOC* works well with $K = 4$, with $K = 2$ it produces poorly organized trajectories: PPOC* tends to make each option broad and stochastic, because random movement can cover more targets than two narrow behaviors (Fig. 5d). In contrast, TASRO trajectories generated under the same current option can differ depending on which option was active before: rows fix the previous option k^- , and columns fix the current option k (Fig. 5e). This gives the controller extra flexibility.

Quantitatively, the same effect appears on the AntMaze training task: PPOC* with $K = 2$ stays at 15.2%, while TASRO reaches 54.4% with $K = 2$ (Fig. 4e). This is consistent with the Markov-option design: the agent has learned how to switch from one option to another, leading to a stable walking behaviors that resemble rotations in different directions (Fig. 5e).

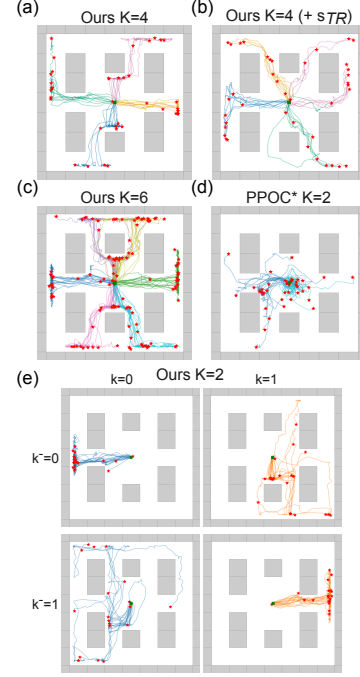


Figure 5: Training-task AntMaze option visualizations.

4 Conclusion and limitations

We introduced TASRO to test whether frozen-option transfer improves when options encode reusable task-agnostic behaviors and the controller handles goal-conditioned composition. The main finding is that TASRO is the only frozen-library method in our experiments that avoids a degenerate option library across all four domain families. This supports a specific view of HRL: option discovery should optimize for recomposability under a new controller, not only for training-task performance. TASRO implements this view through information asymmetry, a responsibility-weighted regularizer, a sticky prior, and a fully option-switching Markovian structure. Unlike prior option-augmented frameworks that keep reselection independent of the previously active option, TASRO makes switching itself a Markov decision on (s_t, k_{t-1}) , so both continuing and switching can depend on the current state and previous option. The transfer results back this design across navigation, locomotion, and manipulation, with the largest gains in harder settings where reuse demands both stable low-level behavior and flexible recomposition. The ablations confirm the mechanism: hiding task-relevant information from the options prevents them from absorbing the training task, and conditioning on the previously active option produces more organized behavior under small option libraries.

TASRO is not limited to frozen reuse: since it learns options and controller jointly, the same machinery could support downstream fine-tuning, as in HiPPO [14]. We chose frozen transfer because it more strictly tests whether learned low-level behaviors are reusable without modification; frozen reuse and unrestricted downstream learning answer different questions. The central limitation is choosing

the single training task. Frozen-option transfer can succeed only when that task exposes behaviors useful downstream. AntMaze and HumanoidMaze use closely related training and downstream tasks; Panda and Barkour test less aligned transfer. TASRO still transfers in Panda, but Barkour shows that a compact library trained on a fixed route can leave important downstream behaviors uncovered. The main open problem is therefore how to choose or design a training task for a downstream task family; related questions are how to choose K when the downstream behavioral range is unknown and how to set stickiness automatically when both too little and too much persistence can hurt learning. Overall, TASRO points toward HRL systems in which low-level behaviors are pretrained for recomposability, inspected separately, and reused through new goal-conditioned controllers.

References

- [1] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2): 181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1.
- [2] Peter Dayan and Geoffrey E. Hinton. Feudal reinforcement learning. In *Advances in Neural Information Processing Systems 5 (NIPS)*, volume 5, pages 271–278. Morgan-Kaufmann, 1992. URL <https://proceedings.neurips.cc/paper/1992/hash/d14220ee66aee73c49038385428ec4c-Abstract.html>.
- [3] Thomas G. Dietterich. Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000. doi: 10.1613/jair.639.
- [4] Matthew M. Botvinick, Yael Niv, and Andrew G. Barto. Hierarchically organized behavior and its neural foundations: A reinforcement-learning perspective. *Cognition*, 113(3):262–280, 2009. doi: 10.1016/j.cognition.2008.08.011.
- [5] Timothy A. Mann and Shie Mannor. Scaling up approximate value iteration with options: Better policies with fewer iterations. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, volume 32 of *Proceedings of Machine Learning Research*, pages 127–135. PMLR, 2014. URL <https://proceedings.mlr.press/v32/mann14.html>.
- [6] George Konidaris and Andrew G. Barto. Building portable options: Skill transfer in reinforcement learning. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 895–900, 2007. URL <https://www.ijcai.org/Proceedings/07/Papers/144.pdf>.
- [7] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=SJx63jRqFm>.
- [8] Kevin Frans, Jonathan Ho, Xi Chen, Pieter Abbeel, and John Schulman. Meta learning shared hierarchies. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://openreview.net/forum?id=SyX0IeWAW>.
- [9] Francisco M. Garcia, Chris Nota, and Philip S. Thomas. Learning reusable options for multi-task reinforcement learning, 2020. URL <https://arxiv.org/abs/2001.01577>.
- [10] Maximilian Igl, Andrew Gambardella, Jinke He, Nantas Nardelli, N. Siddharth, Wendelin Böhmer, and Shimon Whiteson. Multitask soft option learning. In *Proceedings of the Thirty-Sixth Conference on Uncertainty in Artificial Intelligence (UAI)*, volume 124 of *Proceedings of Machine Learning Research*, pages 969–978. PMLR, 2020. URL <https://proceedings.mlr.press/v124/igl20a.html>.
- [11] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, volume 31, pages 1726–1734, 2017. doi: 10.1609/aaai.v31i1.10916.
- [12] Martin Klissarov, Pierre-Luc Bacon, Jean Harb, and Doina Precup. Learning options end-to-end for continuous action tasks, 2017. URL <https://arxiv.org/abs/1712.00004>.

- [13] Christian Daniel, Gerhard Neumann, and Jan Peters. Hierarchical relative entropy policy search. In *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 22 of *Proceedings of Machine Learning Research*, pages 273–281. PMLR, 2012. URL <https://proceedings.mlr.press/v22/daniel12.html>.
- [14] Alexander C. Li, Carlos Florensa, Ignasi Clavera, and Pieter Abbeel. Sub-policy adaptation for hierarchical reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=ByeWogStDS>.
- [15] Vivek Veeriah, Tom Zahavy, Matteo Hessel, Zhongwen Xu, Junhyuk Oh, Iurii Kemaev, Hado van Hasselt, David Silver, and Satinder Singh. Discovery of options via meta-learned subgoals. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 29861–29873, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/fa246d0262c3925617b0c72bb20eeb1d-Abstract.html>.
- [16] Xue Bin Peng, Michael Chang, Grace Zhang, Pieter Abbeel, and Sergey Levine. MCP: Learning composable hierarchical control with multiplicative compositional policies. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 3681–3692, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/95192c98732387165bf8e396c0f2dad2-Abstract.html>.
- [17] Ahmed H. Qureshi, Jacob J. Johnson, Yuzhe Qin, Taylor Henderson, Byron Boots, and Michael C. Yip. Composing task-agnostic policies with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=H1ezFREtWH>.
- [18] Seohong Park, Oleh Rybkin, and Sergey Levine. METRA: Scalable unsupervised RL with metric-aware abstraction. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=c5pwL0Soay>.
- [19] Carlos Florensa, Yan Duan, and Pieter Abbeel. Stochastic neural networks for hierarchical reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://openreview.net/forum?id=B1oK8aoxe>.
- [20] Yamen Habib, Dmytro Grytskyy, and Rubén Moreno-Bote. Unsupervised action-policy quantization via maximum entropy mixture policies with minimum entropy components. In *18th European Workshop on Reinforcement Learning (EWRL)*, 2025. URL <https://openreview.net/forum?id=HhbHw2yInZ>. EWRL 2025 poster.
- [21] Nicolas Heess, Greg Wayne, Yuval Tassa, Timothy Lillicrap, Martin Riedmiller, and David Silver. Learning and transfer of modulated locomotor controllers, 2016. URL <https://arxiv.org/abs/1610.05182>.
- [22] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- [23] Vijay R. Konda and John N. Tsitsiklis. On actor-critic algorithms. *SIAM Journal on Control and Optimization*, 42(4):1143–1166, 2003. doi: 10.1137/S0363012901385691.
- [24] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [25] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596. PMLR, 2018. URL <https://proceedings.mlr.press/v80/fujimoto18a.html>.

- [26] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1509.02971>.
- [27] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1352–1361. PMLR, 2017. URL <https://proceedings.mlr.press/v70/haarnoja17a.html>.
- [28] Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. MuJoCo menagerie: A collection of high-quality simulation models for MuJoCo. GitHub repository, 2022. URL http://github.com/google-deepmind/mujoco_menagerie.
- [29] Ken Caluwaerts, Atil Iscen, J. Chase Kew, Wenhao Yu, Tingnan Zhang, Daniel Freeman, Kuang-Huei Lee, Lisa Lee, Stefano Saliceti, Vincent Zhuang, Nathan Batchelor, Steven Bohez, Federico Casarini, Jose Enrique Chen, Omar Cortes, Erwin Coumans, Adil Dostmohamed, Gabriel Dulac-Arnold, Alejandro Escontrela, Erik Frey, Roland Hafner, Deepali Jain, Bauyrjan Jyenis, Yuheng Kuang, Edward Lee, Linda Luu, Ofir Nachum, Ken Oslund, Jason Powell, Diego Reyes, Francesco Romano, Feresteh Sadeghi, Ron Sloat, Baruch Tabanpour, Daniel Zheng, Michael Neunert, Raia Hadsell, Nicolas Heess, Francesco Nori, Jeff Seto, Carolina Parada, Vikas Sindhwani, Vincent Vanhoucke, and Jie Tan. Barkour: Benchmarking animal-level agility with quadruped robots, 2023. URL <https://arxiv.org/abs/2305.14654>.
- [30] Alexandre Galashov, Siddhant M. Jayakumar, Leonard Hasenclever, Dhruva Tirumala, Jonathan Schwarz, Guillaume Desjardins, Wojciech M. Czarnecki, Yee Whye Teh, Razvan Pascanu, and Nicolas Heess. Information asymmetry in KL-regularized RL. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=S11qMn05Ym>.
- [31] Dhruva Tirumala, Alexandre Galashov, Hyeonwoo Noh, Leonard Hasenclever, Razvan Pascanu, Jonathan Schwarz, Guillaume Desjardins, Wojciech Marian Czarnecki, Arun Ahuja, Yee Whye Teh, and Nicolas Heess. Behavior priors for efficient reinforcement learning. *Journal of Machine Learning Research*, 23(221):1–68, 2022. URL <https://jmlr.org/papers/v23/20-1038.html>.
- [32] Sasha Salter, Kristian Hartikainen, Walter Goodwin, and Ingmar Posner. Priors, hierarchy, and information asymmetry for skill transfer in reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=0v4VkCSkHNm>.
- [33] Jesse Zhang, Haonan Yu, and Wei Xu. Hierarchical reinforcement learning by discovering intrinsic options. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=r-gPPHEjpmw>.
- [34] Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1312–1320. PMLR, 2015. URL <https://proceedings.mlr.press/v37/schaul15.html>.
- [35] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hind-sight experience replay. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 5048–5058, 2017. URL <https://papers.nips.cc/paper/2017/hash/453fadbd8a1a3af50a9df4df899537b5-Abstract.html>.
- [36] Seohong Park, Jongwook Choi, Jaekyeom Kim, Honglak Lee, and Gunhee Kim. Lipschitz-constrained unsupervised skill discovery. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=BGvt0ghNgA>.

- [37] Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. Dynamics-aware unsupervised discovery of skills. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=HJgLZR4KvH>.
- [38] Christian Daniel, Herke van Hoof, Jan Peters, and Gerhard Neumann. Probabilistic inference for determining options in reinforcement learning. *Machine Learning*, 104(2–3):337–357, 2016. doi: 10.1007/s10994-016-5580-x.
- [39] Michał Bortkiewicz, Władysław Pałucki, Vivek Myers, Tadeusz Dziarmaga, Tomasz Arczewski, Łukasz Kuciński, and Benjamin Eysenbach. Accelerating goal-conditioned RL algorithms and research. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2408.11052>.
- [40] Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992. doi: 10.1137/0330046.

A Appendix

A.1 Related work

Prior work on temporal abstraction has made it possible to learn reusable behavior libraries, but it has left open a specific design question that matters for whether those behaviors remain useful after transfer: what task information should low-level options be allowed to observe? Existing methods have explored three broad responses to this question: some leave low-level inputs largely unrestricted, some restrict low-level information to promote reusable behavior, and others remove task reward entirely during skill pretraining.

The options framework formalizes temporally extended behavior through intra-option policies, a policy over options, and termination rules [1]. End-to-end option learning, including the option-critic architecture and its continuous-action extensions, trains these components jointly to maximize task reward [11, 12]. What these methods leave unconstrained is the input design of the low level: by default, option policies may observe the same goals or task-defining state variables available to the rest of the agent. This makes it easy for the low level to encode task-specific solutions directly, so the hierarchy does not itself protect option reuse.

Work on portable or shared options targets transfer more explicitly, but often through agent-centric structure, multi-task data, or other sources of cross-task signal rather than through low-level input restriction within a single source problem. Portable options use agent-centric representations and initiation sets to make skills transportable across tasks [6]. Other methods assume access to task distributions, multi-task data, or trajectories from previously solved tasks and learn shared option structure from that crosstask signal [10, 15, 9]. MSOL [10], in particular, learns transferable soft options in a multitask setting, with evaluation in discrete-action domains.

Restricted low-level information has also been studied more broadly through information asymmetry, where a lower-level or prior policy is intentionally given a narrower observation than the task-facing controller [30–32]. Within hierarchical control, the closest structural relatives are methods that also separate goal-conditioned composition from lower-level control. MCP freezes task-agnostic motor primitives pretrained from motion-capture imitation and relearns a goal-conditioned composer at transfer time, but it composes primitives multiplicatively at each primitive step rather than learning temporally extended options from a single source reinforcement-learning problem [16]. Composing Task-Agnostic Policies assumes a given set of pretrained task-agnostic primitive policies and learns to compose them for new tasks, but it does not discover the primitive library jointly from a single source reinforcement-learning problem [17]. HiPPo uses masked lower-level sub-policies but fine-tunes both hierarchy levels on downstream tasks rather than treating the low level as a frozen reusable library [14]. HIDIO is also close in motivation, but it learns task-agnostic options as continuous latent codes rather than as a small discrete option library, and it does not evaluate frozen-option transfer with a newly trained controller [33]. Our training task is goal-conditioned in the standard reinforcement-learning sense [34, 35], but the distinguishing choice is not goal conditioning itself; it is restricting that signal to the high-level controller and critic while studying frozen reusable discrete options from a single source problem.

A more radical response to the same problem is to remove task reward entirely during pretraining. DIAYN maximizes mutual information between skill identity and visited states to induce behavioral diversity [7]; LSD adds a Lipschitz constraint to encourage more dynamic skills [36]; DADS emphasizes skills with predictable outcomes that support downstream planning [37]; and METRA uses metric-aware latent abstractions to scale unsupervised skill discovery [18]. These methods can produce broad behavioral coverage, but the learned behaviors are not optimized for any particular downstream task during pretraining. Our method keeps task reward, but restricts where task information is allowed to enter the hierarchy.

Responsibility-weighted updates arise naturally when option identity is treated as a latent variable. Probabilistic option inference treats option identity and termination as latent variables, recovers option components from trajectories with EM-style inference, and extends the same framework to reinforcement learning through weighted policy-search updates [38]. HiREPS similarly learns hierarchical mixtures through responsibility-weighted updates under a KL trust region [13]. Our learning rule shares this broad responsibility-weighted, KL-regularized lineage, but uses a different control model: switch-aware per-step reselection with a sticky prior in place of explicit termination inference or generic one-step mixture assignments.

Each of these families addresses part of the challenge but leaves a specific aspect unresolved: end-to-end option learning leaves low-level inputs unconstrained; transfer-oriented methods rely on external primitive sources, multi-task structure, action-space composition, or different transfer regimes; unsupervised skill discovery removes task reward altogether; and probabilistic methods provide the optimization framework without enforcing the same structural input split.

A.2 Derivations and Update Details

This section derives the TASRO update equations from the regularized Bellman equation in Eq. 6. We first rewrite the local Bellman maximization problem at a fixed option-augmented state as a controller objective plus one objective for each option. We then derive the exact fixed- Q , fixed-responsibility targets for the option policies and the controller, show how the frozen-responsibility update defines a local surrogate, and finally explain how the practical algorithm approximates the continuous action integrals in the closed-form targets with Monte Carlo sampling.

All derivations are stated for a fixed goal g . To keep notation light, we omit g from conditioning arguments in this section. Thus $w(k|x)$, $Q(x, k, a)$, $\rho(k|x, a)$, and $\pi_k^*(a|x)$ denote the goal-conditioned quantities $w(k|x, g)$, $Q(x, k, a, g)$, $\rho(k|x, a, g)$, and $\pi_k^*(a|x, g)$. The closed-form option target is derived at the option-augmented state x , using these quantities. The information asymmetry enters only when the neural option policy is fit: the weighted likelihood evaluates the parametric option as $\pi_{\theta, k}(a|x)$. The derivation describes one policy-improvement step: hold the critic Q and responsibility ρ fixed, update the option policies, and then update the controller using the updated option scores.

Local fixed- Q objective. Fix an option-augmented state $x = (s, k^-)$ and a critic $Q(x, k, a)$. The local objective optimized by the policy-improvement step is the Bellman improvement term corresponding to Eq. 6:

$$\mathcal{J}_x(\pi, w) = R(x; \pi, w) + \sum_{k=1}^K w(k|x) \int da \pi_k(a|x) Q(x, k, a). \quad (14)$$

Here $\pi = \{\pi_1, \dots, \pi_K\}$ is the option library and w is the controller. The critic is held fixed in this policy-improvement step; critic learning is handled separately by the Bellman regression described in Section 2.2.

Responsibility identity. The mixture policy is

$$\pi_m(a|x) = \sum_{j=1}^K w(j|x) \pi_j(a|x). \quad (15)$$

For any action a with $\pi_m(a|x) > 0$, the responsibility of option k is

$$\rho(k|x, a) = \frac{w(k|x) \pi_k(a|x)}{\pi_m(a|x)}. \quad (16)$$

Equivalently,

$$\pi_m(a|x) \rho(k|x, a) = w(k|x) \pi_k(a|x). \quad (17)$$

This identity converts expectations under the mixture policy into controller-weighted expectations under individual options and is used throughout the derivation.

Mixture-entropy decomposition. Using Eq. 17, the mixture entropy can be written in terms of controller entropy, option entropies, and responsibility entropy:

$$\mathcal{H}(\pi_m(\cdot|x)) = \mathcal{H}(w(\cdot|x)) + \sum_{k=1}^K w(k|x) \mathcal{H}(\pi_k(\cdot|x)) - \mathbb{E}_{a \sim \pi_m(\cdot|x)} [\mathcal{H}(\rho(\cdot|x, a))]. \quad (18)$$

The action entropies are differential entropies over the continuous action space, while $\mathcal{H}(w(\cdot|x))$ and $\mathcal{H}(\rho(\cdot|x, a))$ are discrete entropies over options.

Effective coefficients. After applying the mixture-entropy identity, the regularizer is governed by three effective coefficients:

$$\phi_1 := \alpha - \beta, \quad \phi_2 := \alpha + \nu, \quad \phi_3 := \alpha - \eta + \lambda. \quad (19)$$

The coefficient ϕ_1 controls option action-level softness: larger ϕ_1 gives softer option targets, while smaller ϕ_1 makes options more sharply value-seeking. The coefficient ϕ_2 controls responsibility-based specialization: larger ϕ_2 pulls an option more strongly toward actions already attributed to that option. The coefficient ϕ_3 controls controller softness: larger ϕ_3 spreads probability more broadly across options, while smaller ϕ_3 makes the controller more sharply committed to a single option. We assume $\phi_1 > 0$, $\phi_2 \geq 0$, and $\phi_3 > 0$. Under these assumptions, the option and controller subproblems are concave after Q and ρ are fixed, and the frozen-responsibility bound below is nonnegative. The next proposition uses these coefficients to separate the local fixed- Q , fixed-responsibility objective into a controller term and one term for each option.

Objective decomposition. Substituting Eq. 18 into the regularizer $R(x; \pi, w)$ in Eq. 3 groups all terms into either a controller-only term or an option-local term. Here $\pi_k(\cdot|x)$ denotes the target distribution for option k at the option-augmented state x ; the masked parametric policy $\pi_{\theta,k}(\cdot|x_P)$ appears later only in the sampled weighted-likelihood fit.

Proposition 1 (Decomposition of the regularized Bellman objective). *For fixed option-action values $Q(x, k, a)$ and fixed responsibility $\rho(k|x, a)$, the local objective in Eq. 14 decomposes as*

$$\mathcal{J}_x(\pi, w) = \mathcal{L}_w(w|x) + \sum_{k=1}^K w(k|x) \mathcal{L}_k^{\rho}(\pi_k|x), \quad (20)$$

where the controller term is

$$\mathcal{L}_w(w|x) = \phi_3 \mathcal{H}(w(\cdot|x)) + \lambda \sum_{k=1}^K w(k|x) \log w_{\text{prior}}(k|x), \quad (21)$$

and the option-local term for option k is

$$\mathcal{L}_k^{\rho}(\pi_k|x) = \phi_1 \mathcal{H}(\pi_k(\cdot|x)) + \int da \pi_k(a|x) [Q(x, k, a) + \phi_2 \log \rho(k|x, a)]. \quad (22)$$

Proof. Write $w_k = w(k|x)$, $\pi_k(a) = \pi_k(a|x)$, and $\rho_k(a) = \rho(k|x, a)$. Since $\pi_m(a|x) = \sum_j w_j \pi_j(a)$, the responsibility identity gives

$$\pi_m(a|x) \rho_k(a) = w_k \pi_k(a).$$

For points where $w_k \pi_k(a) > 0$, this implies

$$\log \pi_m(a|x) = \log w_k + \log \pi_k(a) - \log \rho_k(a).$$

Points where $w_k \pi_k(a) = 0$ contribute zero to the integrals below under the standard convention $0 \log 0 = 0$. Multiplying by $w_k \pi_k(a)$, integrating over actions, and summing over options gives

$$\begin{aligned} \mathcal{H}(\pi_m(\cdot|x)) &= - \sum_k w_k \int da \pi_k(a) \log \pi_m(a|x) \\ &= - \sum_k w_k \log w_k - \sum_k w_k \int da \pi_k(a) \log \pi_k(a) + \sum_k w_k \int da \pi_k(a) \log \rho_k(a). \end{aligned}$$

The first term is $\mathcal{H}(w(\cdot|x))$, the second term is $\sum_k w_k \mathcal{H}(\pi_k(\cdot|x))$, and the last term is $-\mathbb{E}_{a \sim \pi_m(\cdot|x)}[\mathcal{H}(\rho(\cdot|x, a))]$, because

$$\mathbb{E}_{a \sim \pi_m}[\mathcal{H}(\rho(\cdot|x, a))] = - \sum_k w_k \int da \pi_k(a) \log \rho_k(a).$$

This proves Eq. 18.

Substituting this identity into Eq. 3, before expanding the KL term, gives

$$R(x; \pi, w) = (\alpha - \eta) \mathcal{H}(w(\cdot|x)) + (\alpha - \beta) \sum_k w_k \mathcal{H}(\pi_k(\cdot|x)) \\ - (\alpha + \nu) \mathbb{E}_{a \sim \pi_m} [\mathcal{H}(\rho(\cdot|x, a))] - \lambda D_{\text{KL}}(w(\cdot|x) \| w_{\text{prior}}(\cdot|x)).$$

Expanding the KL term then gives

$$R(x; \pi, w) = (\alpha - \eta + \lambda) \mathcal{H}(w(\cdot|x)) + \lambda \sum_k w_k \log w_{\text{prior}}(k|x) \\ + (\alpha - \beta) \sum_k w_k \mathcal{H}(\pi_k(\cdot|x)) - (\alpha + \nu) \mathbb{E}_{a \sim \pi_m} [\mathcal{H}(\rho(\cdot|x, a))],$$

where we used

$$-\lambda D_{\text{KL}}(w(\cdot|x) \| w_{\text{prior}}(\cdot|x)) = \lambda \mathcal{H}(w(\cdot|x)) + \lambda \sum_k w_k \log w_{\text{prior}}(k|x).$$

Using the responsibility identity once more,

$$-\mathbb{E}_{a \sim \pi_m} [\mathcal{H}(\rho(\cdot|x, a))] = \sum_k w_k \int da \pi_k(a|x) \log \rho(k|x, a).$$

Adding the fixed- Q term from Eq. 14 and collecting coefficients with $\phi_1 = \alpha - \beta$, $\phi_2 = \alpha + \nu$, and $\phi_3 = \alpha - \eta + \lambda$ yields Eq. 20 with the terms in Eqs. 21 and 22. $\square$

Closed-form option target. For fixed Q , fixed ρ , and $\phi_1 > 0$, the option-local objective in Eq. 22 is maximized by the density

$$\pi_k^*(a|x) = \frac{\rho(k|x, a)^{\phi_2/\phi_1} \exp(Q(x, k, a)/\phi_1)}{\int da' \rho(k|x, a')^{\phi_2/\phi_1} \exp(Q(x, k, a')/\phi_1)}. \quad (23)$$

To see this, maximize Eq. 22 over densities $\pi_k(\cdot|x)$ subject to $\int da \pi_k(a|x) = 1$. The Lagrangian is

$$\mathfrak{L}(\pi_k, \zeta) = -\phi_1 \int da \pi_k(a|x) \log \pi_k(a|x) \\ + \int da \pi_k(a|x) [Q(x, k, a) + \phi_2 \log \rho(k|x, a)] \\ + \zeta \left(\int da \pi_k(a|x) - 1 \right).$$

The stationarity condition with respect to $\pi_k(a|x)$ is

$$-\phi_1 (\log \pi_k(a|x) + 1) + Q(x, k, a) + \phi_2 \log \rho(k|x, a) + \zeta = 0.$$

Solving for the maximizing density gives

$$\pi_k^*(a|x) \propto \rho(k|x, a)^{\phi_2/\phi_1} \exp(Q(x, k, a)/\phi_1),$$

and normalizing over actions gives Eq. 23. This target is defined at the option-augmented state x . The practical option update then fits $\pi_{\theta, k}(\cdot|x_P)$ by weighted likelihood.

Closed-form controller target. After maximizing $\mathcal{L}_k^\rho(\pi_k|x)$ for each option, the controller sub-problem depends on option k only through the achieved value of this term. Define this value as

$$M_k(x) := \mathcal{L}_k^\rho(\pi_k^*|x). \quad (24)$$

Equivalently,

$$M_k(x) = \phi_1 \mathcal{H}(\pi_k^*(\cdot|x)) + \mathbb{E}_{a \sim \pi_k^*(\cdot|x)} [Q(x, k, a) + \phi_2 \log \rho(k|x, a)]. \quad (25)$$

This score uses the same fixed responsibility ρ as the option update; responsibility is not recomputed inside this policy-improvement step. The only remaining optimization variable is the controller $w(\cdot|x)$, which maximizes

$$\max_w \phi_3 \mathcal{H}(w(\cdot|x)) + \lambda \sum_k w(k|x) \log w_{\text{prior}}(k|x) + \sum_k w(k|x) M_k(x). \quad (26)$$

For $\phi_3 > 0$, the solution is

$$w^*(k|x) = \frac{\exp(M_k(x)/\phi_3)w_{\text{prior}}(k|x)^{\lambda/\phi_3}}{\sum_j \exp(M_j(x)/\phi_3)w_{\text{prior}}(j|x)^{\lambda/\phi_3}}. \quad (27)$$

This follows by optimizing Eq. 26 over the option simplex. The Lagrangian is

$$\begin{aligned} \mathfrak{L}(w, \xi) = & -\phi_3 \sum_k w(k|x) \log w(k|x) + \lambda \sum_k w(k|x) \log w_{\text{prior}}(k|x) \\ & + \sum_k w(k|x) M_k(x) + \xi \left(\sum_k w(k|x) - 1 \right). \end{aligned}$$

For each option k , stationarity gives

$$-\phi_3 (\log w(k|x) + 1) + \lambda \log w_{\text{prior}}(k|x) + M_k(x) + \xi = 0.$$

Thus

$$w(k|x) \propto \exp(M_k(x)/\phi_3)w_{\text{prior}}(k|x)^{\lambda/\phi_3},$$

and normalizing over options gives Eq. 27. Thus the controller selects options according to their scores $M_k(x)$, while the sticky prior increases the probability of reselecting the previous option through the factor $w_{\text{prior}}(k|x)^{\lambda/\phi_3}$.

Surrogate improvement with frozen responsibilities. The fixed-responsibility update can be viewed as optimizing a surrogate for the local objective. Let $\rho^{\text{old}}(k|x, a)$ be a reference responsibility with full support over options, and define

$$\tilde{\mathcal{J}}_x^{\rho^{\text{old}}}(\pi, w) := \mathcal{L}_w(w|x) + \sum_k w(k|x) \mathcal{L}_k^{\rho^{\text{old}}}(\pi_k|x), \quad (28)$$

where $\mathcal{L}_k^{\rho^{\text{old}}}$ is Eq. 22 with ρ replaced by ρ^{old} .

Proposition 2 (Frozen-responsibility lower bound). *Assume $\phi_1 > 0$ and $\phi_2 \geq 0$. For any candidate hierarchy (π, w) , let $\rho^{\pi, w}$ be the responsibility induced by that hierarchy. Then*

$$\mathcal{J}_x(\pi, w) - \tilde{\mathcal{J}}_x^{\rho^{\text{old}}}(\pi, w) = \phi_2 \mathbb{E}_{a \sim \pi_m(\cdot|x)} [D_{\text{KL}}(\rho^{\pi, w}(\cdot|x, a) \parallel \rho^{\text{old}}(\cdot|x, a))] \geq 0. \quad (29)$$

The bound is tight when the candidate hierarchy induces the frozen responsibility on the support of $\pi_m(\cdot|x)$.

Proof. The only difference between $\mathcal{J}_x$ and $\tilde{\mathcal{J}}_x^{\rho^{\text{old}}}$ is the responsibility term in Eq. 22. Using $\pi_m(a|x)\rho^{\pi, w}(k|x, a) = w(k|x)\pi_k(a|x)$, the difference is

$$\begin{aligned} & \phi_2 \sum_k w(k|x) \int da \pi_k(a|x) [\log \rho^{\pi, w}(k|x, a) - \log \rho^{\text{old}}(k|x, a)] \\ & = \phi_2 \int da \pi_m(a|x) \sum_k \rho^{\pi, w}(k|x, a) \log \frac{\rho^{\pi, w}(k|x, a)}{\rho^{\text{old}}(k|x, a)}, \end{aligned}$$

which is Eq. 29. The inequality follows from nonnegativity of KL divergence and $\phi_2 \geq 0$. $\square$

Choose ρ^{old} as the responsibility induced by the current hierarchy $(\pi^{\text{old}}, w^{\text{old}})$. The surrogate is then tight at the current hierarchy:

$$\tilde{\mathcal{J}}_x^{\rho^{\text{old}}}(\pi^{\text{old}}, w^{\text{old}}) = \mathcal{J}_x(\pi^{\text{old}}, w^{\text{old}}).$$

In the finite-action tabular case, let $(\pi^{\text{new}}, w^{\text{new}})$ be the exact maximizer of the frozen surrogate, obtained by the closed-form option target in Eq. 23 followed by the controller target in Eq. 27. Combining the lower bound with exact surrogate maximization gives

$$\begin{aligned} \mathcal{J}_x(\pi^{\text{new}}, w^{\text{new}}) & \geq \tilde{\mathcal{J}}_x^{\rho^{\text{old}}}(\pi^{\text{new}}, w^{\text{new}}) \\ & \geq \tilde{\mathcal{J}}_x^{\rho^{\text{old}}}(\pi^{\text{old}}, w^{\text{old}}) = \mathcal{J}_x(\pi^{\text{old}}, w^{\text{old}}). \end{aligned}$$

Thus, in the finite-action tabular case, one exact update cannot decrease the local fixed- Q objective $\mathcal{J}_x$. This statement is for the exact finite-action update at fixed Q and fixed x . The practical update uses continuous actions, sampled action candidates, neural option policies and controllers, target networks, and replay. The result therefore explains the structure of the update, but it is not a monotonic-improvement guarantee for the implemented algorithm.

Sampled approximation. The exact option target in Eq. 23 is a density over continuous actions. The practical update replaces this density by a weighted set of sampled candidate actions. As above, the appendix suppresses the fixed goal argument g ; in the implementation, the sampled quantities are evaluated at (x, g) .

Sampled option update. Bars denote target networks: $\bar{w}$ is the target controller, $\bar{\pi}_k$ are the target options, $\bar{\pi}_m$ is the corresponding mixture policy, and $\bar{Q}_i$ are the two target critics used in clipped double- Q evaluation. Let

$$\bar{\rho}(k|x, a) = \frac{\bar{w}(k|x)\bar{\pi}_k(a|x_P)}{\bar{\pi}_m(a|x)}$$

be the responsibility computed under the target networks. Concretely, $\bar{\rho}$ is computed at the start of each policy-improvement step and held fixed while the sampled option and controller targets are formed.

The exact option target in Eq. 23 is proportional to

$$\rho(k|x, a)^{\phi_2/\phi_1} \exp(Q(x, k, a)/\phi_1).$$

The sampled update estimates this target using target-network quantities and samples from $\bar{\pi}_k(\cdot|x_P)$. With $a_k^{(n)} \sim \bar{\pi}_k(\cdot|x_P)$, the practical algorithm assigns the importance-corrected weight

$$\exp b_k^{(n)} = \frac{\bar{\rho}(k|x, a_k^{(n)})^{\phi_2/\phi_1} \exp(\min_i \bar{Q}_i(x, k, a_k^{(n)})/\phi_1)}{\bar{\pi}_k(a_k^{(n)}|x_P)}. \quad (30)$$

Normalizing these weights over the N candidates gives

$$\omega_k^{(n)} = \frac{\exp b_k^{(n)}}{\sum_{m=1}^N \exp b_k^{(m)}}. \quad (31)$$

The option network is then trained by weighted maximum likelihood, using the same weights but evaluating the learned option on the option state x_P :

$$\mathcal{L}_{\pi_k} = -\mathbb{E}_{\mathcal{D}} \left[\sum_{n=1}^N \omega_k^{(n)} \log \pi_{\theta, k}(a_k^{(n)}|x_P) \right]. \quad (32)$$

The total option-policy loss sums over options:

$$\mathcal{L}_{\text{opt}} = \sum_{k=1}^K \mathcal{L}_{\pi_k}. \quad (33)$$

Sampled controller score. The same sampled weights also estimate the score used by the controller. To see this, define the normalizer of the exact option target:

$$F_k(x) := \int da \rho(k|x, a)^{\phi_2/\phi_1} \exp(Q(x, k, a)/\phi_1). \quad (34)$$

Then Eq. 23 can be written as

$$\pi_k^*(a|x) = \frac{1}{F_k(x)} \rho(k|x, a)^{\phi_2/\phi_1} \exp(Q(x, k, a)/\phi_1).$$

Taking logs and rearranging gives

$$Q(x, k, a) + \phi_2 \log \rho(k|x, a) = \phi_1 \log \pi_k^*(a|x) + \phi_1 \log F_k(x).$$

Substituting this identity into Eq. 22 with $\pi_k = \pi_k^*$ gives

$$\begin{aligned} M_k(x) &= \phi_1 \mathcal{H}(\pi_k^*(\cdot|x)) + \int da \pi_k^*(a|x) [\phi_1 \log \pi_k^*(a|x) + \phi_1 \log F_k(x)] \\ &= \phi_1 \mathcal{H}(\pi_k^*(\cdot|x)) - \phi_1 \mathcal{H}(\pi_k^*(\cdot|x)) + \phi_1 \log F_k(x) = \phi_1 \log F_k(x). \end{aligned}$$

The second line uses $\int da \pi_k^* \log \pi_k^* = -\mathcal{H}(\pi_k^*)$, and the fact that $F_k(x)$ does not depend on a . Thus estimating $F_k(x)$ gives the option score $M_k(x) = \phi_1 \log F_k(x)$. This is the score used by the controller target, which has the form

$$w^*(k|x) \propto \exp(M_k(x)/\phi_3) w_{\text{prior}}(k|x)^{\lambda/\phi_3}.$$

Since $\exp b_k^{(n)}$ is the importance-corrected integrand for estimating $F_k(x)$ under the target networks, we use

$$\widehat{M}_k(x) = \phi_1 \log \left(\frac{1}{N} \sum_{n=1}^N \exp b_k^{(n)} \right). \quad (35)$$

After applying the logarithm, $\widehat{M}_k(x)$ is a consistent sampled proxy for $\phi_1 \log F_k(x)$, but not an unbiased finite-sample estimate.

Replacing $M_k(x)$ by $\widehat{M}_k(x)$ in Eq. 27 gives the sampled controller target

$$\widehat{w}^*(k|x) = \frac{\exp(\widehat{M}_k(x)/\phi_3) w_{\text{prior}}(k|x)^{\lambda/\phi_3}}{\sum_j \exp(\widehat{M}_j(x)/\phi_3) w_{\text{prior}}(j|x)^{\lambda/\phi_3}}. \quad (36)$$

The controller network is trained to match this sampled target by minimizing

$$\mathcal{L}_{\text{ctrl}} = \mathbb{E}_{\mathcal{D}} [D_{\text{KL}}(\widehat{w}^*(\cdot|x) \| w_\psi(\cdot|x))]. \quad (37)$$

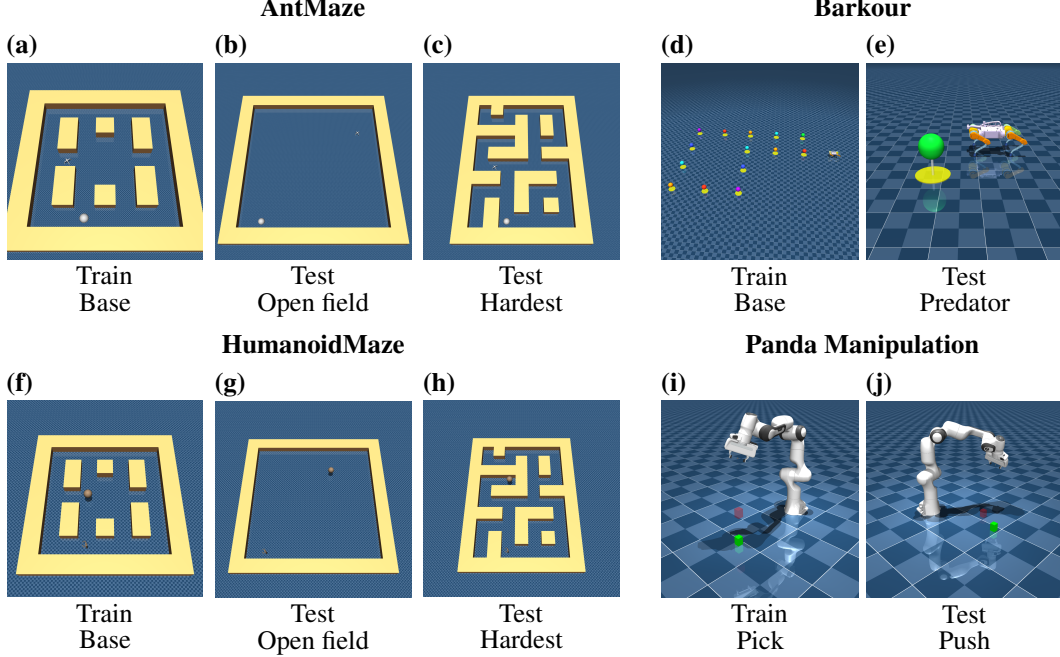


Figure 6: Source and target environments used in the transfer experiments. Within each domain, the left panel is the source training task and the remaining panels are target transfer tasks. The top row shows AntMaze and Barkour; the bottom row shows HumanoidMaze and Panda manipulation.

A.3 Environment and transfer details

All experiments use the same frozen-option transfer protocol. During training-task learning, the option policies $\pi_{\theta,k}$, controller w_ψ , and critics Q_{ξ_i} are trained jointly on a single training task. During downstream transfer, we freeze the learned option library and discard the training-task controller and critics. We then train a new downstream controller, critic, and temperature parameter on each downstream task. The downstream controller and critic receive the option-augmented state and goal (x, g) , while each frozen option receives only the masked option input x_p . Thus options never receive the goal directly, and under the masking used in this paper they also do not observe the task-defining achieved-goal coordinates. The four domains below instantiate this protocol in navigation, locomotion, and manipulation.

A.3.1 AntMaze navigation

AntMaze [39] is a quadruped navigation domain. The source task is simple-maze navigation, and the transfer tasks are open-field and hardest-maze goal reaching (Fig. 6a–c). The environment state s is 29-dimensional, and the goal g is the target xy position in world coordinates.

The controller observes the full augmented state $x = (s, k^-)$. Options act on $x_p = (s_p, k^-)$, where s_p is the 27-dimensional proprioceptive state with global xy position removed. For transfer, each frozen option library is reused by training a new controller and critic on each downstream layout.

Source and transfer episodes are both 1000 steps. The downstream controller selects an option every $H = 10$ primitive steps, long enough to produce measurable displacement but short enough for frequent route corrections. The dense reward is $20\Delta d + 10\mathbf{1}_{\text{success}} - 0.5\|a\|^2$, where Δd is progress toward the xy goal. Success is declared when the xy distance to the goal is below 0.5 meters. Episodes terminate on success or loss of balance, defined by torso height outside $[0.2, 1.0]$ or a flipped torso orientation.

A.3.2 HumanoidMaze navigation

HumanoidMaze [39] follows the same frozen-transfer protocol as AntMaze, but with higher-dimensional observations and less stable humanoid dynamics. The source task is simple-maze

navigation (Fig. 6f), and the transfer tasks are open-field and hardest-maze goal reaching (Fig. 6g,h). The environment state s is 51-dimensional, consisting of 26 generalized coordinates and 25 generalized velocities; the extra position coordinate comes from the floating-base quaternion. The goal g is a target xyz position in world coordinates with fixed height $z = 1.25$.

The controller observes the full augmented state $x = (s, k^-)$. Options act on $x_P = (s_P, k^-)$, where s_P is the 49-dimensional proprioceptive state with global xy position removed. For transfer, each frozen option library is reused by training a new controller and critic on each downstream layout.

Source and transfer episodes are both 1000 steps. The downstream controller selects an option every $H = 10$ primitive steps. The dense reward is $300\Delta d + 101_{\text{success}} - 0.1\|a\|^2$, where Δd is progress toward the 3D target. Success is declared when the 3D target distance is below 0.5 meters. Episodes terminate on success or loss of balance, defined by torso height outside $[1.0, 2.0]$.

A.3.3 Barkour waypoint locomotion

Barkour is a quadruped waypoint-navigation domain based on the Google Barkour model [28, 29]. The source task uses a fixed base route, and the transfer task uses the predator waypoint layout (Fig. 6d–e). The environment state s is 48-dimensional, the goal g is the current waypoint xy position in world coordinates, and the action is a 12-dimensional joint-target command.

In the source task, the robot follows a fixed 14-waypoint route (Fig. 6d). In the predator transfer task, there is no fixed route. Whenever the robot reaches the current waypoint, defined as entering a 0.15-meter radius, a new waypoint is sampled relative to the robot’s current pose. In our transfer configuration, the new waypoint is sampled uniformly at radius 2 meters, with direction drawn from a $\pm 170^\circ$ cone around the robot’s forward yaw. This allows almost all headings except a narrow rear sector. The source route is a simplified waypoint-only mimic of the original Barkour agility setting [29]: it keeps the Barkour robot, but removes the richer terrain, obstacle-course structure, and external kick disturbances.

The controller observes the full augmented state $x = (s, k^-)$ and the current waypoint goal. Options act on $x_P = (s_P, k^-)$, where s_P removes the robot’s global xy position, leaving a 46-dimensional locomotion state. This option state contains global yaw, local velocity, angular rates, projected gravity, joint angles, joint velocities, and the previous action.

Source episodes are 1000 steps, and predator transfer episodes are 4000 steps. The longer transfer horizon accommodates the 10-waypoint random sequence. During downstream transfer, each frozen option library is reused by training a new controller and critic on the predator layout. The downstream controller selects an option every $H = 10$ primitive steps. The waypoint reward uses weights 10.0 for clipped progress, 0.2 for heading alignment, 20.0 for intermediate waypoint reaches, 100.0 for final success, 100.0 for proximity shaping, and -1.0 for falling; velocity-alignment, upright, action-magnitude, and action-rate terms are disabled in this setting. Episodes terminate on success after reaching 10 predator waypoints or on physical failure, defined by the torso flipping, any joint leaving its valid range, or torso height falling below 0.18 meters. This transfer task exposes waypoint directions much broader than the fixed source route.

A.3.4 Panda manipulation transfer

Panda manipulation tests frozen-option transfer between two arm-manipulation objectives. The source task is single-phase pick-and-orient, and the transfer task is table pushing (Fig. 6i–j). Both tasks use the same 47-dimensional environment state s and an 8-dimensional accumulated delta joint-target command: 7 arm joints and 1 gripper command. The policy outputs a joint-target delta, and the environment integrates these deltas into the commanded joint targets. The state contains cube position (3), cube orientation as a 6D rotation representation (6), robot joint positions (9), robot joint velocities (9), gripper position (3), gripper orientation as a 6D rotation representation (6), cube–grripper relative position (3), and control residuals (8), for a total of 47 dimensions.

In the source pick-and-orient task, the robot must grasp a cube from the table and match a target pose in free space. The goal g is 9-dimensional: target cube position (3) and target cube orientation in a 6D rotation representation (6). In the transfer task table pushing, the robot must push the cube to a target position on the table. The goal g is the 3-dimensional target cube position. Thus the full pick observation has dimension $47 + 9 = 56$, while the full push observation has dimension $47 + 3 = 50$.

The goal dimensionality differs across tasks, but the frozen options remain unchanged because they observe neither goal.

The controller observes the full augmented state $x = (s, k^-)$ and the task goal. Options act on $x_P = (s_P, k^-)$, where s_P is the same 26-dimensional proprio-control state in both tasks: 9 robot joint positions, 9 robot joint velocities, and 8 control residual features. The control residual is the difference between the commanded joint targets and the current robot joint positions. Thus the options do not see the cube pose, gripper pose, cube–grripper relative position, or goal. This identical option observation across source and transfer tasks enables frozen reuse.

Source episodes are 200 steps, and push transfer episodes are 300 steps. During downstream transfer, each frozen option library is reused by training a new controller and critic on table pushing. The downstream controller selects an option every $H = 5$ primitive steps, shorter than navigation horizons because manipulation episodes are shorter and require faster contact-level corrections. The pick reward weights are 4.0 for gripper-to-cube approach, 8.0 for cube-to-target pose matching, 0.25 for avoiding floor collision, and 0.3 for mild joint regularization. The push reward weights are 4.0 for gripper-to-cube approach, 8.0 for cube progress toward the target, and 0.25 for avoiding floor collision. Source success is declared when $\|p_{\text{box}} - p_{\text{target}}\| + 0.25\|R_{\text{box}}^{6D} - R_{\text{target}}^{6D}\| < 0.05$. Push success is declared when the cube xy distance to the target is below 0.05 meters. Success does not terminate an episode; episodes terminate only on workspace exit or simulator instability.

Existing assets and licenses. Our simulated robot assets are based on MuJoCo Menagerie [28]. In particular, the Franka Emika Panda model and the Google Barkour v0 model are distributed under the Apache License 2.0 in MuJoCo Menagerie. The AntMaze and HumanoidMaze environments are based on the JaxGCRL maze environments [39], which are also distributed under the Apache License 2.0. We cite the Barkour benchmark setup [29]; all third-party assets are used only for simulated control experiments.

A.4 Training and hyperparameter details

A.4.1 Source TASRO training

Table 1 summarizes the training settings used for TASRO in each domain family. The table includes the domain-specific option-view dimension, while the transfer descriptions above report the transfer-time option duration H . The domain-specific regularization coefficients are listed separately in Table 2.

Table 1: TASRO source-training settings by domain family. “Option dim” is the masked state dimension seen by the option policies after removing the task-defining coordinates described in Section A.3.

Domain	K	Option dim	Steps	Batch	Ep. len	$\text{lr}_{Q/\pi/w}$
AntMaze	4	27	100M	512	1000	3×10^{-4}
HumanoidMaze	4	49	200M	512	1000	10^{-3}
Barkour	3	46	200M	256	1000	10^{-3}
Panda pick	16	26	350M	256	200	10^{-3}

Table 2: TASRO objective coefficients used in training-task learning.

Domain	α	β	η	ν	λ	Ω_{stick}	γ
AntMaze	0.6	0.1	0.4	0.1	0.4	2.0	0.99
HumanoidMaze	0.6	0.1	0.4	0.1	0.4	6.0	0.99
Barkour	0.4	0.1	0.1	0.3	0.3	8.0	0.99
Panda pick	0.4	0.2	0.2	0.3	0.3	4.0	0.97

Training uses the replay-based actor-critic implementation with a minimum replay fill of 100k transitions. The maximum replay size is 10M for AntMaze and Panda, 5M for HumanoidMaze, and 20M for Barkour. Soft target updates use Polyak averaging [40] with coefficient 0.995.

A.4.2 Downstream transfer training

In frozen-option transfer, the option library learned on the training task is kept fixed. For each downstream task, we discard the training-task controller and critic and train a new controller, critic, and temperature parameter. Within each domain, all frozen option libraries are evaluated with the same downstream training setup, so performance differences reflect how reusable the frozen options are.

The downstream training settings are summarized in Tab. 3. In all rows, the new controller and critic observe the option-augmented state and goal (x, g) , while each frozen option keeps the same option input used during training-task learning. The critic learning rate is 3×10^{-4} , and the soft target coefficient is 0.005 in all downstream runs. The soft-Q temperature is initialized at $\tau_{\text{sq}} = 1.0$ and tuned automatically during downstream training.

Table 3: Downstream frozen-option transfer settings. The option library is fixed; only a new controller, critic, and temperature parameter are trained.

Domain	Downstream task	Steps	Batch	Ep. len	H
AntMaze	Open field	200k	256	1000	10
AntMaze	Hardest maze	2M	256	1000	10
HumanoidMaze	Open field	1M	256	1000	10
HumanoidMaze	Hardest maze	2M	256	1000	10
Barkour	Predator	20M	256	4000	10
Panda	Push	25M	256	300	5

A.5 Baseline source settings

For baseline comparisons, we match the number of options K to the TASRO setting in each domain and evaluate all frozen option libraries with the same domain-specific transfer learner described above. Among the compared methods, only TASRO conditions the low-level options on the previous option index k^- ; the baseline option policies follow their original interfaces. Table 4 summarizes the baseline source-training budgets.

Table 4: Baseline source-training budgets in environment steps. A dash means that the baseline is not included for that domain.

Method	AntMaze	HumanoidMaze	Barkour	Panda
OC	100M	50M	200M	–
PPOC	100M	50M	200M	–
PPOC*	100M	50M	200M	100M
DIAYN	100M	50M	50M	50M
METRA	100M	15M	15M	15M
MSOL	150M	150M	100M	100M

These budgets were selected after preliminary sweeps and correspond to runs where source-training metrics had largely plateaued.

Option-Critic (OC), deep off-policy variant [11]. We use a replay-based deep Option-Critic variant with learned intra-option policies and terminations. OC is included as an option-learning baseline in which the low-level options are exposed to full task information during training-task learning: they receive the state and goal, including current position coordinates. OC is not included in the Panda push comparison. Relative to the original online Option-Critic algorithm, this continuous-control implementation adds replay, soft target networks, and squashed-Gaussian option policies. We compare against this deep off-policy variant rather than the original online algorithm.

PPOC and PPOC* [12]. PPOC and PPOC* share the same on-policy PPO-style option learner and differ only in the option observation. PPOC is the task-exposed option-learning baseline: its low-level options receive the full state and goal during training-task learning. PPOC* masks task information from the low-level options, so the options observe only task-agnostic proprioceptive

state s_P . This tests whether masking alone is enough for frozen-option transfer, without TASRO’s previous-option conditioning, sticky prior, or responsibility-based specialization. OC and PPOC are not evaluated on Panda push because their options are goal-conditioned during source training. The Panda source and transfer goals have different structure: pick-and-orient uses a free-space 9D pose goal, whereas push uses a table-level 3D position goal. This goal-distribution shift makes OC/PPOC a poor frozen-option transfer comparison for the pick-to-push setting.

DIAYN and METRA [7, 18]. DIAYN and METRA are unsupervised skill-discovery baselines. They ignore the training-task reward and learn goal-free, state-conditioned skills. DIAYN encourages skills to visit distinguishable states, while METRA encourages coverage using metric structure over the state space. We train both methods with SAC [22].

MSOL [10]. MSOL is a frozen-option transfer method originally evaluated in discrete-action domains with options learned from multiple tasks. We adapt it to continuous control and use the same single training-task protocol as the other methods. The adapted implementation uses PPO-style option learning with minibatch size 1024, $\text{lr}_{\text{posterior}} = 3 \times 10^{-4}$, $\text{lr}_{\text{prior}} = 10^{-5}$, and $\gamma = 0.99$. After training-task learning, we freeze the goal-free prior option library.

A.6 Compute resources

As representative AntMaze runs, source training and downstream transfer were executed on a single workstation with one NVIDIA GeForce RTX 5090 GPU (34.2 GB GPU memory), 16 physical CPU cores (32 logical threads), 132.6 GB system RAM, and CUDA 13.0. Training AntMaze options for 100M environment steps used 128 parallel environments, batch size 512, 1000-step episodes, and $K = 4$ options; one run took 11,899 seconds, or about 3.31 GPU-hours. The AntMaze open-field transfer setting in Fig. 2c used 200k environment steps, 10 parallel environments, batch size 256, 1000-step episodes, and transfer option duration $H = 10$; one run took 1,357 seconds, or about 0.38 GPU-hours. The full project, including reported runs, baseline training, ablations, and preliminary sweeps not included in the paper, consumed approximately 800 single-GPU hours in total.

A.7 Additional qualitative forced-option plots

Figure 7 shows the remaining four AntMaze open-field seeds for each method beyond the single representative seed used in the main paper. This appendix figure is intended to expose seed-to-seed variation in the qualitative option structure.

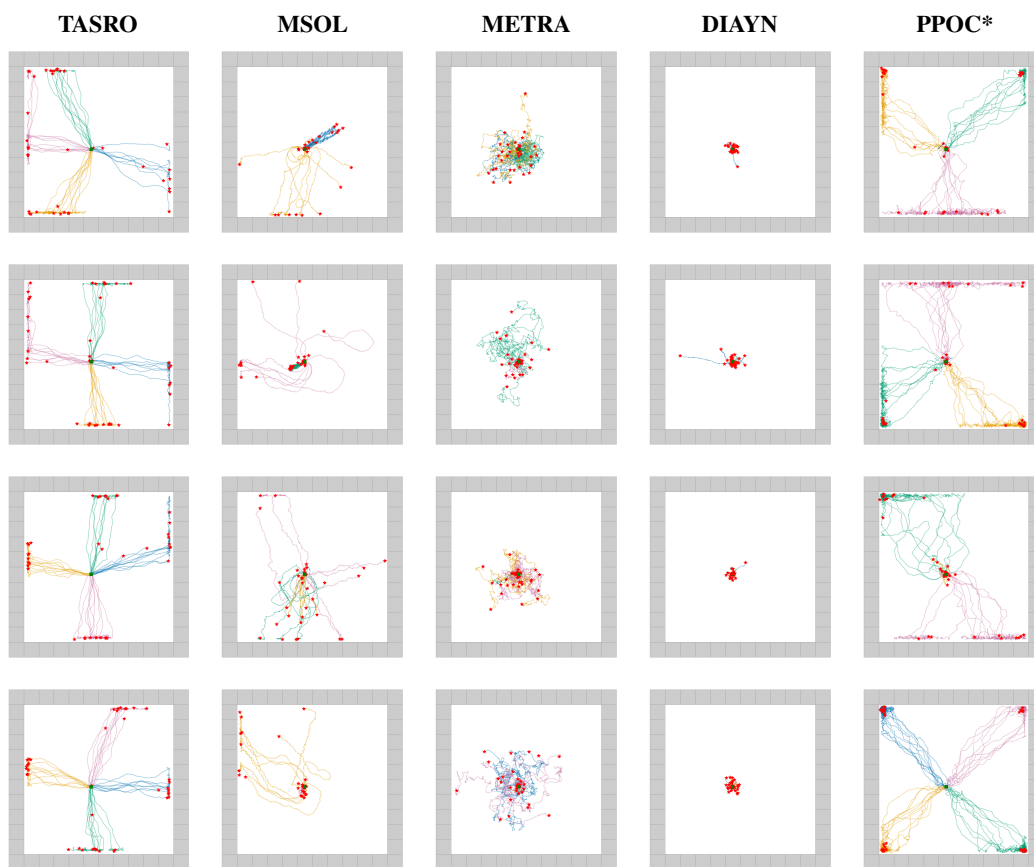


Figure 7: Additional AntMaze forced-option visualizations in the open-field transfer environment. Columns correspond to methods, and rows show the four additional seeds not included in Figure 3.